

Overview

Perception, planning, and navigation form the autonomy stack for mobile robots. A robot must infer where it is, understand free/occupied space, plan a feasible path, and execute motion safely under uncertainty.

High-level loop:

1. sense (camera/lidar/imu/encoders),
2. localize/map,
3. plan global route,
4. compute local control actions,
5. monitor and recover when needed.

Perception

Sensor Modalities

- Cameras: rich semantic information, sensitive to lighting.
- LiDAR: geometric depth structure, robust for obstacle shape.
- IMU: short-term orientation/acceleration estimation.
- Wheel encoders: odometry with drift over time.

Practical autonomy uses sensor fusion because no single sensor is reliable in all conditions.

Vision Tasks Reused from Lesson 9

Robotics perception often reuses:

- object detection for obstacle/asset identification,
- segmentation for drivable-space estimation,
- pose estimation for docking/manipulation.

Mapping & Localization

SLAM (Intuition)

SLAM means building a map while estimating robot pose within that map, simultaneously.

Why it is hard:

- map quality depends on pose estimates,
- pose estimates depend on map quality.

Common outputs include occupancy grids where each cell is free, occupied, or unknown.

Occupancy Grids and Localization

- Occupancy grid: discretized map used for collision-aware planning.
- Localization: estimate pose relative to map frame (for example map $\rightarrow$ base_link transformations in ROS-style systems).

Planning & Navigation

Graph-Based Search (A*)

A* finds a least-cost path on graph/grid using:

$$f(n) = g(n) + h(n)$$

- $g(n)$: cost from start to node n .
- $h(n)$: heuristic estimate from n to goal.

With admissible heuristic, A* is optimal on the graph.

Reactive vs Deliberative Navigation

- Deliberative: computes global paths (longer horizon).
- Reactive: quickly handles local dynamic obstacles.

Production stacks combine both: global planner + local controller + recovery behaviors.

Safety and Collision Avoidance

Essential controls:

- velocity limits,
- safety buffers/costmaps,
- emergency stop conditions,
- fallback/recovery actions when planning fails.

Business Case Studies & Exceptions

Case Study 1: Autonomous Delivery Robots

Scenario:

- Campus/last-mile robots must navigate sidewalks and crossings.

Constraints:

- dynamic pedestrians, weather changes, intermittent GPS.

System pattern:

- SLAM/localization + global route planner + reactive local obstacle avoidance.

Case Study 2: Smart Factory AMRs

Scenario:

- Robots move materials between workstations.

Constraints:

- narrow aisles, humans/forklifts, uptime requirements.

Practical pattern:

- prebuilt maps plus continuous localization refinement,
- traffic-manager layer for fleet conflict resolution.

Exceptions

- In highly structured static environments, simple line-following or waypoint control may beat full SLAM complexity.
- For severe sensor degradation (dust, glare), fallback teleoperation may be safer than autonomous mode.

Interview Questions & Answers

1. **Explain SLAM in simple language.**
The robot builds a map and estimates its own location at the same time.
2. **What is an occupancy grid?**
A cell-based map indicating free, occupied, and unknown regions for planning.
3. **What is A* used for in robotics?**
Finding low-cost paths from start to goal on map graphs.
4. **Why do robots use multiple sensors?**
Different sensors fail differently; fusion improves robustness.
5. **What is localization drift?**
Accumulated pose error over time, common with pure odometry.
6. **Global planner vs local planner?**
Global planner gives route; local planner generates near-term safe motions.
7. **How do perception errors affect planning?**
Wrong obstacle/pose estimates can produce unsafe or infeasible paths.
8. **What is a recovery behavior?**
Fallback action when robot gets stuck, e.g., replan, backtrack, rotate.
9. **Why are costmaps important?**
They encode obstacle risk and clearance for local control.
10. **When is full autonomy unnecessary?**
In constrained repetitive routes where deterministic guidance is enough.
11. **What role do behavior trees play in navigation stacks?**
They orchestrate planners/controllers/recoveries into robust workflows.
12. **How do you validate navigation safety?**
Simulation stress tests, scenario replay, hardware-in-loop checks, and runtime safety guards.
13. **What is the biggest deployment challenge in mobile robotics?**
Handling domain shift from controlled tests to dynamic real-world scenes.
14. **Why does latency matter in navigation loops?**
Slow loops react too late to obstacles and destabilize motion.
15. **How would you debug repeated path failures?**
Inspect map quality, localization confidence, planner parameters, and local obstacle layers.

References

- Nav2 documentation (architecture and behavior trees): <https://docs.nav2.org/>
- Nav2 concepts: <https://docs.nav2.org/concepts/index.html>
- ROS2 SLAM Toolbox package docs: https://docs.ros.org/en/ros2_packages/humble/api/slam_toolbox/index.html
- NVIDIA DLI Edge AI/Robotics syllabus (navigation modules): <https://developer.nvidia.com/edge-ai-robotics-teaching-kit-syllabus>
- Berkeley CS285 (planning/control context via RL): <https://www2.eecs.berkeley.edu/Courses/CS285/>